INFORME TÉCNICO: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++

Institución: E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Materia: Laboratorio de Programación (LPR)

Curso: 5° Año 3° División

Docentes: Prof. York & Prof. Yamil

Fecha de Entrega: 17 de Septiembre de 2026

1. Introducción y Objetivos

El presente informe detalla el desarrollo de la Actividad 4, centrada en el análisis y comprensión de la gestión de memoria de bajo nivel en la arquitectura de un computador. El objetivo principal consistió en implementar un sistema de control de acceso por edad en C++, interactuando de manera directa con los casilleros de la memoria RAM mediante el uso de punteros y operaciones de desreferenciación.

2. Modalidad y Entorno Híbrido (Proyecto Integrador / P.I.A.)

- **Formato de Entrega y Modalidad Grupal:** La actividad se desarrolló bajo la modalidad grupal (Proyecto Integrador - P.I.A.), cumpliendo estrictamente con los estándares formales del formato APA v7 en PDF y asegurando su almacenamiento local bajo la ruta y el nombre obligatorio Docs/InformeProyectoCADENASPUNTEROS.pdf.
- **Integrantes del Grupo y Distribución de Tareas:**
  - **Ferrari Ulises:** Diseño de la estructura del repositorio en Git, configuración del entorno de desarrollo local con VS Code y MinGW, y redacción general de la documentación.
  - **Maximo Orue:** Análisis de la lógica de punteros, validación de direcciones de memoria en formato hexadecimal y pruebas de consola del código base.
  - **Del Pino Lucas:** Implementación del código fuente principal (src/main.cpp) y resolución de operadores de desreferenciación (* y ->).
  - **Mattia Federico:** Elaboración del esquema gráfico de la memoria RAM (Capturas/esquema_ram.png) y desarrollo de las respuestas teóricas del cuestionario de control.
- **Entorno de Compilación y Pruebas:** Se empleó un entorno híbrido combinando la terminal integrada de Visual Studio Code con el

compilador de MinGW (g++) de forma local, complementado con el control de versiones y repositorio oficial en GitHub estructurado según los requerimientos de la cátedra.

3. Descripción del Código y Funcionamiento en la RAM

El programa desarrollado (`src/main.cpp`) declara una variable entera `edad` inicializada en 0 y un puntero de tipo entero `int* p = &edad;`. Mediante el operador de dirección (`&`), el puntero almacena la ubicación física exacta del casillero de memoria asignado a la variable.

Durante la ejecución, la captura de datos se realiza desreferenciando el puntero con `cin >> *p;`, lo que garantiza que el valor ingresado por el usuario se almacene directamente en el espacio de memoria original sin duplicaciones innecesarias.

4. Cuestionario Técnico de Control

Pregunta 1: El Prototipo y Diferencia entre `&` y `*`

- **Explicación:** El operador ampersand (`&edad`) extrae y devuelve la dirección física absoluta en la RAM donde se aloja la variable, mientras que el asterisco acoplado al puntero (`*p`) efectúa una desreferenciación para acceder al contenido almacenado en el interior de dicho casillero de memoria.
- **Respuesta:** La instrucción `cout << p;` imprime en la consola la dirección de memoria hexadecimal física (por ejemplo, `0x61fe1c`), indicando el casillero exacto de la RAM. En contraparte, la instrucción `cout << *p;` imprime el valor numérico entero que se encuentra guardado dentro de ese casillero (en este caso, la edad ingresada por el usuario).

Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Omitir el operador de desreferenciación al intentar capturar datos mediante la entrada estándar (`cin >> p;`) genera un conflicto tipográfico y de seguridad en el sistema.
- **Respuesta:** Es estrictamente obligatorio desreferenciar el puntero utilizando `*p` porque, de lo contrario, el compilador intentaría sobreescribir la dirección de memoria hexadecimal que el puntero almacena en lugar de modificar el valor de la variable original. Esto ocasiona un error crítico de compilación o fallas graves de segmentación en tiempo de ejecución.

Pregunta 3: Formato Hexadecimal en la Memoria

- **Explicación:** Las direcciones físicas de la memoria RAM se gestionan a nivel de hardware utilizando bases numéricas óptimas para la representación binaria agrupada.
- **Respuesta:** El sistema operativo emplea la notación hexadecimal (base 16, identificada con el prefijo `0x`) porque permite compactar y simplificar la lectura de secuencias binarias extensas. Facilita enormemente el trabajo de los programadores y depuradores al representar de manera limpia y estandarizada los bytes de la memoria en comparación con el sistema decimal tradicional.

Ferrari Ulises. (2026). GitHub.
https://github.com/Ferraribip/Act4.git